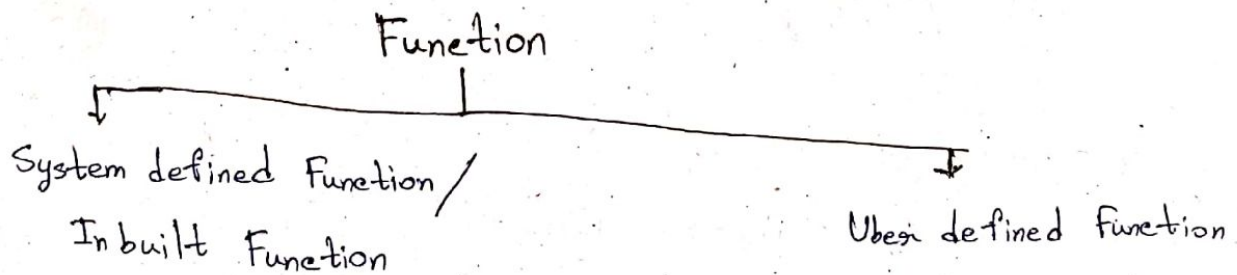


Function - It is a basically a datatype which can be used to reuse the same thing again and again without defining the same thing multiple number of times.

Features of Function :-

- i) Reuseability - It is a property within function which can be used ^{with out} to write the same code again and again.
- ii) Code-complexity - If we write any programme as a breakup of function then programming logic can be more precise ~~or~~ to the user or user can easily understand the code without much of complexity.
- iii) Error Connection - By using function programme can be ~~div~~ divided into smaller parts so in that case to check the error and rectify the error can be easier ~~to~~ for the user.
- iv) Time-Complexity - ~~A~~ By using function user can rectify the error less time than the normal program so time complexity can be ~~messe~~ measured by using function.



System defined Function - It is a function which can be created by the C developer (Dennis Ritchie).

example - printf(), scanf(), main(), getch()

User defined Function - It is a function which can be used by the user / can be created by the user.

Example - add(), multi(), div(), subtract(),

Structure :-

- i) Function declaration - It must be declared before the main function.
- ii) Function definition - It is always ^{after the} end of the main function that means body of the function should be defined after the end of the bracket of main().
- iii) Function Calling - Calling must be present within the main function.

User defined Function Structure Example -

void add(); // function declaration -

```
void main()
{
    add(); // calling
    getch();
}

void add() // function definition
{
    c = a + b;
    printf("%d", c);
}
```

Syntax of function :-

<u>void</u>	<u>add</u>	<u>()</u>
return	name	parameter/
type of	of the	argument
function	function	of a function.

Based on return type and parameter passing 4 types of functions are there -

i) No return type No parameter passing -

<u>example -</u>	<u>void</u>	<u>add()</u>
	return	parameter
	type	passing

ii) No parameter passing but with return type -

<u>example -</u>	<u>int.</u>	<u>add()</u>	function or no return calculation
	return	parameter	example 10, Print function - as
	type	passing	return no return no , Value is is is

iii) No return type but with parameter passing -

<u>example -</u>	<u>void</u>	<u>add(int, int)</u>	main - no no no , Value is is is Hold
	return	parameter	example 10, Print
	type	passing	example 10, Print

iv) With both return type and parameter passing -

<u>example -</u>	<u>int</u>	<u>add(int, int)</u>	example 10, Print
	return	parameter	example 10, Print
	type	passing	example 10, Print

What is call by value?

When we are passing value or passing variables value as an argument at the time of function calling it is known as call by value.

example -

```
void add (int, int);  
void main ( )  
{  
    int a, b;  
    printf("Enter 2 No. =");  
    scanf("%d %d", &a, &b);
```

Call by value → add(a, b) // calling
getch();
}
void add (int x, int y)
{
 printf("The addition is %d", x+y);
}

What is ~~not~~ call by reference?

When we are calling a function at that time we are passing an address of any variable and the function will get the variables value using parameterised pointer this concept is known call by reference.

```
void name (char *);  
void main ( )  
{  
    char a[10];  
    printf("Enter the string =");  
    gets(a);  
    name(&a[0]) → call by reference  
    getch();  
}  
void name(char *p)  
{  
    for(; *p != '\0'; p++)  
        printf("%c", *p);  
}
```

call by value.

1) When we are passing any value as an actual parameter at the time of function calling to the calling function is known as call by value.

2) call by value is logical.

3) ~~It is~~ It is logical because if we are passing value to the function at the time of function calling.

4) Example -

```
void add (int, int);  
void main ( )  
{  
    int a, b;  
    printf ("Enter 2 No. = ");  
    scanf ("%d %d", &a, &b);  
    add (a, b);  
    getch ( );  
}  
void add (int x, int y);  
{  
    printf ("The addition is %d", x+y);  
}
```

call by reference

1) When we are passing any address of a variable as an actual parameter at the time of function calling to the calling function is known as call by reference.

2) call by reference is physical.

3) It is physical because we are passing address to the function at the time of ~~add~~ function calling.

4) Example -

```
void areasq (int *x);  
void main ( )  
{  
    int x;  
    printf ("Enter the No. = ");  
    scanf ("%d", &x);  
    areasq (&x);  
    getch ( );  
}  
void areasq (int *y)  
{  
    printf ("Area of Square = %d", *y);  
}
```

Local Variable :- When we are declaring a variable within the scope of any function or main it can be used within the scope only. Outside of the scope of the function or main it can't be accessible it is known as Local Variable.

example -

```
void add ();  
void main ()  
{  
    int x, y;  
    printf("Enter 2 No. : ");  
    scanf("%d %d", &x, &y);  
    getch();  
}  
void add ()  
{  
    printf("%d %d", x, y);  
}
```

local variable

Because x & y variable is local to main function only it can't be accessible outside of the main function.

Global Variable :- When we are declaring a variable outside of the main function as well as scope of any function. It is known as global variable because it can be accessible all of the scope of the function as well as scope of the main function.

```
void add ();  
int x, y; → Global variable.  
void main ()  
{  
    printf("Enter 2 No. : ");  
    scanf("%d %d", &x, &y);  
    getch();  
}  
void add ()  
{  
    printf("%d %d", x, y);  
}
```